Plan: load the member's data once and share it across pages

fix_up_moto | approved plan | 2026-11-11

1. Context

- **What prompted this**
  - The same member data (name, phone, email, points, Riwayat) is shown on Home, Profile, MyPoint and Edit Profile.
  - Today each of those four pages builds its own `HomeBloc` and fetches on mount. Edit Profile fetches again on every push.
  - When something changes the data, `DataRefreshCubit.invalidate(DataKind.stats)` makes three tabs refetch the same record. One change causes three requests.
- **What we found**
  - Nothing dispatches `ProfileLoadRequested`. The pages load their data through `HomeStatsRequested`.
  - Both loaders ask the server for the **same record**:
    - `ProfileRemoteDataSourceImpl.getProfile` posts to `browseTrans` with `Jenis: MEMBERSHIP` (`profile_remote_data_source.dart:47`).
    - `ProfileEntity` (id, name, email, phone, status, active, qty, point) is a strict subset of `DashboardStatsEntity`, which adds `detail` and `detail2`.
  - A second pipeline through `ProfileLoadRequested` would keep two copies of the same fields. After an edit, one copy could be stale.
- **The pattern we already use (Riwayat in MyPoint)**
  - Riwayat makes no request of its own. It reads `stats.detail` from the record that is already loaded (`membership_page.dart:450-453`).
  - The Voucher tab uses a load-once guard, `_loadVouchersOnce` (line 390).
  - This plan applies the same idea to the whole app: one place loads the data, and the pages only read it.
- **Decisions made with the user**
  - Share the **stats record**.
  - Use **one app-scoped holder**, and that holder is `HomeBloc` itself, so the pages' existing `BlocBuilder<HomeBloc, HomeState>` and `context.read<HomeBloc>()` keep working.
  - Do **not** build a separate `ProfileLoadRequested` pipeline.

2. Design rules

- **Read: one noticeboard.** One app-wide `HomeBloc` loads the record. Every page reads it and none fetches on mount.
- **Write: separate and short-lived.** Edit profile, change password and (later) delete stay in page-scoped flows (`ProfileBloc`).
- **After a write: re-read.** The server's reply to an update has no member record (`profile_bloc.dart:50-51`).
  - So a successful write calls `DataRefreshCubit.invalidate(stats)`, and the holder reloads once.
  - No local patching, because the server may reformat values such as the phone number.

- **Refresh in the background.** When data is already on screen, a reload keeps showing the old data until the new data arrives.
  - This is **required**, not polish: a spinner would unmount the Edit Profile form and the member would lose what they typed.
- **Reset on logout.** The holder is cleared when the member signs out, so one account's data never shows for the next.

3. Steps

Step 1: make `HomeBloc` app-scoped

- `lib/core/di/injection_container.dart:267`: change `registerFactory` to `registerLazySingleton`, like `AuthBloc` and `DataRefreshCubit`.
- `lib/app.dart`: add `BlocProvider<HomeBloc>.value(value: sl<HomeBloc>())` to the root providers.
  - Use `.value`, never `create:`, or the singleton gets closed on dispose (the rule in CLAUDE.md).
- Fix the comments that are now wrong:
  - `home_bloc.dart:9` ("Provided at the HomePage level")
  - `app.dart:24`
  - the "Page-scoped BLoCs" list in CLAUDE.md (move `HomeBloc` to the app-scoped group)
  - `profile_page.dart:35-36`, which says `ProfileBloc` waits for the `BrowseMember` endpoint

Step 2: make `HomeBloc` safe to share

- Files: `home_bloc.dart`, `home_event.dart`, `home_state.dart`.
- **Reset event**
  - Add `HomeResetRequested`, which emits `HomeInitial`.
- **Request counter**
  - Each `HomeStatsRequested` takes `final seq = ++_requestSeq;` and drops its result after the `await` if `seq != _requestSeq`.
  - `HomeResetRequested` also bumps the counter.
  - This stops a fetch still running at logout from landing in the next account's state.
  - It also makes the latest of two overlapping requests win.
- **Background refresh**
  - First load (nothing yet): emit `HomeLoading`, as today.
  - Reload while `HomeLoaded`: emit `HomeLoaded(stats, refreshing: true)`, then `HomeLoaded(newStats)` when the answer arrives.
    - `refreshing` is part of the state's equality, so both states always emit.
    - Pages matching `HomeLoaded(:final stats)` keep showing their body, so nothing unmounts.
  - If a background reload fails, keep the old data and emit `HomeLoaded(old)`. (Open item: show a snackbar for the failure? See section 6.)
- **Pull-to-refresh wait**
  - Profile's `_refresh` waits for `HomeLoaded || HomeError` (`profile_page.dart:167-178`).
  - Change the condition to `HomeLoaded && !refreshing`, or it would stop immediately.

Step 3: one `StatsBootstrap` widget at the root

- New file `lib/features/home/presentation/widgets/stats_bootstrap.dart`, modelled on `PushBootstrap` (`core/push/push_bootstrap.dart`).

- Mount it in `app.dart`, in the `MaterialApp.router builder`, below the providers.
- It does three things:
  - on `AuthAuthenticated` (and once at mount if already signed in): dispatch `HomeStatsRequested`
  - on `AuthUnauthenticated`: dispatch `HomeResetRequested`
  - wrap its child in the **single** `RefreshOn(kind: DataKind.stats, ...)` that dispatches `HomeStatsRequested`
- Reused as is, no changes: `DataRefreshCubit`, `RefreshOn`, and the four `invalidate(stats)` callers:
  - `membership_page.dart:88`
  - `my_bikes_page.dart:245`
  - `edit_profile_page.dart:295`
  - `add_bike_page.dart:93`

Step 4: pages stop owning and fetching

- Same edit in four files: delete the `BlocProvider(create: sl<HomeBloc>()..add(HomeStatsRequested()))` and the page-level `RefreshOn`. Keep the `BlocBuilder`, the Retry buttons and the pull-to-refresh dispatches.
- The four files:
  - `home_page.dart`: `HomePage` returns `_HomeView` directly.
  - `profile_page.dart:56-64`: also remove the `buildWhen` guard, which the background refresh makes unnecessary.
  - `membership_page.dart:51-61, 102-107`: keep `RewardsBloc` and `RedeemBloc`, drop only `HomeBloc` and `RefreshOn`.
  - `edit_profile_page.dart:55-62`: keep `ProfileBloc`, drop `HomeBloc`.

Step 5: protect the Edit Profile form

- Check `_EditProfileForm` in `edit_profile_page.dart`: it should fill its fields **once** (in `initState`), not whenever a new state arrives.
- If it re-fills from new data, change it, so a background reload can never overwrite what the member is typing.
- After a successful save, the existing `invalidate(stats)` (`edit_profile_page.dart:295`) updates every page.

Step 6: writes stay as they are (no code change now)

- `ProfileBloc` keeps update profile and change password.
- **Delete** is not wired yet (`profile_page.dart:140-152`). When it is, the flow should reset the holder (`HomeResetRequested`) and sign the member out through `AuthBloc`.

Step 7: decision on `ProfileLoadRequested`

- Leave it unused. Do not dispatch it.
- If a "profile" name is wanted later, build a `ProfileEntity` from the shared record with a small mapper. Do not make a second request.
- Revisit only if the backend adds an endpoint that is actually lighter than the stats record.

4. Tests (mocktail and bloc_test, per CLAUDE.md)

- **New** `test/features/home/presentation/bloc/home_bloc_test.dart`:
  - first load emits Loading then Loaded

  - reload while loaded emits `refreshing: true`, never `HomeLoading`
  - failed background reload keeps the old data
  - reset emits Initial
  - a response arriving after a reset is dropped
  - of two overlapping requests, the latest wins
- **New** test for `StatsBootstrap`:
  - sign-in dispatches a load
  - sign-out dispatches a reset
  - one `invalidate(stats)` dispatches exactly one load
- **Update** the four page tests (`home`, `profile`, `membership`, `edit_profile`):
  - they now register `sl.registerFactory<HomeBloc>(() => homeBloc)`; switch to `BlocProvider<HomeBloc>.value(value: homeBloc)`
  - change "adds an event on mount" assertions to "adds nothing on mount"
- **New** Edit Profile test: a background reload while the member is typing does not change the field text.

5. Verification

- `flutter analyze` is clean.
- `flutter test`: compare with the known baseline of 13 failures (memory: `flutter-test-baseline-failures`) using `git stash`. Do not run `dart format` on a whole directory.
- On the emulator, count the `Home Dashboard Stats` log lines (`home_bloc.dart:25`), one per fetch:
  - login: **one** line (was one per mounted tab)
  - switching Home, MyPoint and Profile tabs: **no** lines
  - opening Edit Profile: **no** lines
  - adding a bike or claiming a voucher: **one** line (was three)
  - pull-to-refresh on each page: one line, and the indicator stops
  - typing in Edit Profile while a reload happens: the typed text stays
  - logging out and in as a different account: the new account's data, with no flash of the old

6. Risks and open items

- **Fresher data only on trigger.** The record reloads on sign-in, on `invalidate(stats)` or on a pull.
  - A change made on the server (for example, points from a completed booking) shows after a pull.
  - The three tabs already behaved this way. Only Edit Profile loses its reload-on-open.
- **Failed background reload is silent** under the current design. Decide whether to show a snackbar.
- **Stream never closes.** `Profile._refresh` waits on the bloc's stream, and a singleton's stream never closes. A pull abandoned by popping or logging out leaves a harmless unfinished future.
- **Wide blast radius.** `HomeBloc` is read by four pages and four test files, so the page and test updates in Step 4 and section 4 should be done together and checked against the baseline.